



Universidad Tecnológica de Panamá

Licenciatura en Desarrollo y Gestión de Software

Desarrollo de Software IX

Grupo 1GS241

Integrantes:

Cesar Bazán

Eriel Ten Su

Christian Valdespino

Diseño de la arquitectura fullstack (backend, API, servidor,
deploy)

Fecha: 17 de abril de 2026

Introducción

Diseñar una aplicación full stack no es solo conectar una pantalla con una base de datos. Implica decidir cómo se comunican los componentes, cómo se protegen los datos, cómo se organiza el servidor y cómo se publica la aplicación para que otras personas la usen. Cuando una API está bien pensada, el frontend trabaja con menos fricción; cuando el servidor está dividido por responsabilidades, el código se mantiene mejor; y cuando el despliegue se planifica bien, el proyecto deja de depender de configuraciones improvisadas.

Para explicar estos puntos se puede tomar como caso una plataforma ficticia de reservas de salas para una universidad. El sistema permitiría crear reservas, consultar horarios disponibles, modificar citas y validar conflictos entre usuarios. Ese tipo de ejemplo ayuda a aterrizar la teoría sin depender de un proyecto concreto. En esta investigación se desarrollan cuatro puntos: diseño de la API, estructura del servidor, Dokploy y Railway. El objetivo es mostrar qué hace valiosa a cada pieza y cómo encajan entre sí en una arquitectura full stack moderna.

Diseño de la API

Una API es un contrato de comunicación entre sistemas. Su diseño debe ser claro, predecible y consistente. En arquitectura REST, lo ideal es pensar en recursos y no en acciones sueltas. Por ejemplo, en una aplicación de reservas los recursos pueden ser salas, horarios, usuarios y reservas. GET se usa para recuperar información, POST para crear recursos nuevos, PUT y PATCH para actualizar, y DELETE para eliminar. Elegir el método correcto no es un detalle menor: determina si una operación es segura, idempotente o cacheable, y también cómo la interpretará el cliente.

El segundo pilar son los códigos de estado. Una respuesta correcta no debería decir simplemente “ok” o “error”, porque eso obliga al frontend a adivinar qué pasó. En cambio, 200 indica éxito general, 201 señala creación de un recurso, 204 confirma que no hay contenido que devolver, 400 avisa que la petición está mal formada, 401 y 403 separan autenticación de autorización, 404 indica que el recurso no existe y 409 sirve para conflictos como una reserva duplicada. Esa semántica reduce ambigüedad y ayuda a depurar problemas más rápido.

La seguridad debe pensarse desde el principio. OWASP recuerda que las APIs son una superficie de ataque importante porque exponen lógica y datos sensibles. Por eso conviene validar todo en el servidor, limitar la frecuencia de solicitudes cuando sea necesario, controlar el acceso a cada ruta y evitar exponer información que no hace falta. En una API de reservas, por ejemplo, no tendría sentido devolver datos internos del sistema si el usuario solo necesita saber si una sala está libre. La documentación también es crucial: sin una descripción clara de rutas, parámetros y respuestas, el equipo termina perdiendo tiempo en pruebas manuales o en suposiciones.

Una estructura útil para el ejemplo de reservas podría incluir `GET /api/salas`, `GET /api/salas/:id`, `POST /api/reservas`, `PATCH /api/reservas/:id` y `DELETE /api/reservas/:id`. Con esa organización, el frontend sabe exactamente cómo consultar disponibilidad, registrar una reserva o cancelar un turno. Además, versionar la API desde el inicio, por ejemplo con `/api/v1`, permite evolucionar el sistema sin romper a los clientes que ya lo consumen. Esto parece exagerado al principio, pero después ahorra muchos dolores de cabeza.

- Usar nombres de recursos en plural y rutas cortas.
- Devolver códigos HTTP precisos para cada caso.
- Validar y sanitizar datos en el servidor.
- Documentar la API con ejemplos de uso.

Estructura del servidor

El servidor es la parte donde vive la lógica del negocio. Para que no se vuelva una mezcla confusa de rutas, consultas y validaciones, conviene separarlo por capas. Una organización común divide el backend en routes, controllers, services, models, middleware y config. Las rutas reciben la petición HTTP y la encaminan; los controladores coordinan la entrada y la salida; los servicios guardan la lógica de negocio; los modelos representan la persistencia; el middleware atiende tareas transversales; y config centraliza variables de entorno y conexiones.

Esa separación ayuda mucho cuando la aplicación crece. Si cambia la forma de una URL, se modifica la ruta. Si cambia una regla de negocio, se toca el servicio. Si se quiere cambiar el mensaje de error o el formato de respuesta, se ajusta el controlador. Así cada pieza tiene una responsabilidad más o menos clara. En un sistema de reservas, por ejemplo, un servicio puede

impedir que dos usuarios tomen la misma sala al mismo tiempo, mientras que el controlador solo se encarga de recibir la solicitud y devolver el resultado adecuado.

Express explica bien esta idea al mostrar que las rutas se asocian a métodos HTTP como GET y POST, y que cada endpoint ejecuta una función cuando coincide con la solicitud. Esa lógica también puede aplicarse en otros frameworks ligeros o en soluciones nativas de Node o Bun. Lo importante no es el nombre del framework, sino la disciplina con la que se distribuye el trabajo. Si todo se coloca en un solo archivo, el backend puede funcionar al principio, pero se vuelve difícil de probar y de mantener.

Una estructura concreta para el ejemplo podría verse así: routes para salas y reservas, services para verificar conflictos de horario, models para representar reservas y salas, middleware para errores y autenticación, y config para la base de datos. Cuando entra una solicitud POST /api/reservas, el servidor valida que la fecha exista, que la sala esté libre y que el usuario tenga permiso. Si todo está en orden, persiste la reserva; si no, responde con un error claro. Esa cadena hace visible la ventaja de una arquitectura ordenada.

```
server/  
├─ routes/  
│   ├─ salas.ts  
│   └─ reservas.ts  
├─ services/  
│   ├─ salaService.ts  
│   └─ reservaService.ts  
├─ models/  
│   ├─ Sala.ts  
│   └─ Reserva.ts  
├─ middleware/  
│   └─ errorHandler.ts  
└─ config/  
    └─ db.ts
```

Dokploy

Dokploy es una plataforma abierta y autohospedable que simplifica el despliegue sin quitarle control al equipo. Su propuesta gira alrededor de Docker y Traefik, lo que permite manejar

aplicaciones y servicios con una configuración bastante flexible. La ventaja principal es el control: el servidor, los puertos, los dominios, los contenedores y las variables de entorno quedan en manos del equipo que administra la instancia.

Eso la vuelve interesante cuando se necesita aprender más sobre infraestructura o cuando el proyecto exige una configuración muy específica. Por ejemplo, una app de reservas desplegada en Dokploy puede tener un contenedor para el backend, otro para la base de datos y reglas personalizadas para el dominio. El proceso suele incluir conectar el repositorio, definir variables de entorno, activar el despliegue automático y revisar logs cuando algo falla. No es la opción más simple, pero sí una de las más transparentes para entender qué está pasando detrás del despliegue.

En contextos académicos, Dokploy también tiene valor didáctico. Permite ver de forma cercana cómo se levantan servicios, cómo se actualizan versiones y cómo se administra un entorno propio. Eso obliga a asumir más responsabilidad, pero a cambio enseña conceptos que después sirven en cualquier servidor real.

- Control total sobre infraestructura y despliegue.
- Uso de Docker y Traefik para flexibilidad.
- Buena opción para aprendizaje y autogestión.
- Requiere más configuración que un servicio administrado.

Railway

Railway representa el enfoque opuesto: una plataforma administrada que busca reducir al mínimo la fricción para publicar una aplicación. Su documentación la describe como un proveedor de nube que ayuda a aprovisionar infraestructura, desarrollar localmente y desplegar con rapidez. En la práctica, eso significa menos tiempo peleando con servidores y más tiempo enfocándose en la aplicación.

Su mayor ventaja es la velocidad. Railway puede detectar proyectos, manejar variables de entorno y conectar despliegues con GitHub de manera bastante directa. Para una app de reservas o una demo universitaria, eso es muy útil porque permite tener una versión funcional en poco

tiempo. La plataforma también facilita la creación de servicios auxiliares, por lo que el equipo no tiene que montar todo a mano desde cero.

La desventaja es que ofrece menos control fino que una solución autohospedada. Sin embargo, eso no siempre es malo: si el objetivo es avanzar rápido, reducir carga operativa o mostrar una entrega funcional, Railway puede ser la mejor opción. En otras palabras, la plataforma no reemplaza el diseño de la arquitectura; solo cambia dónde se concentra el esfuerzo.

- Configuración inicial rápida y simple.
- Integración cómoda con GitHub.
- Ideal para MVPs, prototipos y entregas cortas.
- Menor control que una solución autohospedada.

Comparación y aplicación práctica

La comparación entre Dokploy y Railway no trata de decidir cuál es “mejor” en abstracto, sino cuál conviene según el contexto. Railway simplifica y acelera; Dokploy da más control y cercanía con la infraestructura. Si el equipo necesita publicar una demo rápido, Railway gana por comodidad. Si lo que se quiere es administrar un entorno propio o aprender despliegue en profundidad, Dokploy ofrece una experiencia más completa.

En una app de reservas, ambas opciones son válidas. El backend puede vivir detrás de una API REST, la base de datos puede configurarse aparte y el frontend puede conectarse por medio de variables de entorno. Lo importante es que la arquitectura no dependa de trucos temporales. Si la API está bien diseñada y el servidor está separado por capas, cambiar de Railway a Dokploy, o al revés, no debería exigir rehacer todo el proyecto.

Aspecto	Railway	Dokploy
Facilidad	Muy alta	Alta, pero con más pasos
Control	Medio	Alto
Ideal para	MVPs y demos	Proyectos con autogestión
Curva de aprendizaje	Baja	Media/alta

Para un proyecto universitario, la elección depende del objetivo del equipo. Si lo más importante es entregar rápido y con poco margen de error operativo, Railway resulta muy práctico. Si el grupo quiere practicar administración real de despliegue, Dokploy ofrece más aprendizaje técnico. En ambos casos, la base sigue siendo la misma: una API ordenada, un servidor modular y buenas prácticas de seguridad.

Buenas prácticas transversales

Además de API, servidor y despliegue, hay prácticas que atraviesan todo el sistema. La validación debe ocurrir en el servidor antes de guardar datos. Los errores deben diferenciar problemas del cliente, fallos de negocio y errores internos. Los secretos deben permanecer fuera del código y guardarse en variables de entorno. Y las pruebas deben cubrir reglas críticas como conflictos de reserva, permisos y eliminación de recursos.

También conviene tener logs claros y cierta trazabilidad. Cuando algo falla, no sirve de mucho tener un mensaje genérico; es mejor saber qué ruta se ejecutó, qué datos llegaron y qué parte del flujo rompió la respuesta. Esa disciplina no hace el sistema más vistoso, pero sí más confiable. Muchas veces esas pequeñas decisiones son las que separan un prototipo frágil de una aplicación que realmente puede sostenerse en el tiempo.

- Validar datos en el backend antes de persistirlos.
- Manejar errores con mensajes claros y útiles.
- Guardar secretos en variables de entorno.
- Probar casos críticos con regularidad.

Errores comunes y recomendaciones

Un error muy común en proyectos de este tipo es mezclar demasiada lógica en el controlador. Al principio parece rápido, pero después cualquier cambio obliga a tocar el mismo archivo una y otra vez. Otro fallo frecuente es no definir bien los estados de la aplicación, por ejemplo permitir reservas duplicadas o dejar endpoints que devuelven respuestas distintas sin una razón clara. Eso genera confusión en el frontend y hace que el sistema sea más frágil.

También suele pasar que la seguridad se deja para el final. Se piensa primero en “que funcione” y luego se intenta protegerlo, pero ya con el proyecto encima eso cuesta más trabajo. Es mejor

controlar permisos, validar datos y ocultar secretos desde el principio. A nivel de despliegue, otra recomendación es probar primero en un entorno pequeño antes de mover todo a producción, porque un cambio de variables o de rutas puede romper algo que localmente parecía perfecto.

Además, es importante entender que una API bien diseñada no depende solo de tener rutas funcionando, sino de definir un contrato claro entre el cliente y el servidor. Red Hat explica que el diseño de API consiste en exponer datos y funciones de manera comprensible para que otros sistemas puedan consumirlos sin confusión, y que en REST los recursos se organizan con métodos HTTP y representaciones consistentes. Por eso, una buena documentación, nombres coherentes y respuestas predecibles ayudan a que el frontend trabaje con menos errores y que el sistema sea más fácil de mantener. También es recomendable definir desde el inicio especificaciones como OpenAPI, porque facilitan la comunicación del equipo y ordenan el desarrollo de la API.

Por último, conviene revisar el sistema con una mirada práctica: ¿se entiende qué hace cada endpoint?, ¿el servidor responde siempre igual?, ¿el despliegue se puede repetir sin inventar pasos raros? Si la respuesta a esas preguntas es sí, entonces la arquitectura va por buen camino. Si no, toca simplificarla antes de que crezca mas de la cuenta.

Conclusiones individuales

César Bazán

A mi me quedo claro que una API no se trata solo de poner rutas y ya. Si no esta bien pensada, despues todo se vuelve mas complicado de mantener. Me gustó ver que los codigos HTTP, la validacion y la documentacion no son cosas separadas, sino parte de lo mismo. La verdad, antes yo veía el backend mas como hacer que responda, pero ahora entiendo que hay que cuidar mucho la forma en que se comunica con el frontend. Siento que eso hace la diferencia entre una app que no funciona bien y una que si se puede usar sin tantos problemas.

Christian Valdespino

Para mi el tema mas importante fue entender que el backend no debe verse como un solo bloque. Dividirlo en rutas, controladores, servicios y modelos ayuda mucho a no enredarse cuando el proyecto crece. Tambien me quedo la idea de que el despliegue importa tanto como el codigo, porque no ayuda tener algo que funciona si luego subirlo se vuelve muy complicado. Yo si creo que Railway y Dokploy sirven para cosas distintas, y eso esta bien; lo importante es saber cual usar segun lo que ocupa el proyecto y el tiempo que tengas.

Eriel Ten Su

Yo pienso que lo mas valioso de este tema es que te obliga a pensar mas ordenado. Antes yo a veces escribia la logica y luego ya miraba como acomodarla, pero aqui se nota que desde el inicio conviene decidir como va a comunicarse todo. Tambien me parecio importante la parte de seguridad, porque uno suele dejarla al final y luego salen fallas que pudieron evitarse.

Conclusión General

comprendimos que desarrollar una API no se trata solo de hacer que funcione, sino de pensar desde el inicio en su diseño, organización y en cómo se comunicará correctamente con el frontend. Ahora vemos el backend como una estructura que debe estar bien dividida en rutas, controladores, servicios y modelos, lo que facilita el mantenimiento y evita problemas cuando el proyecto crece. También entendimos que elementos como los códigos HTTP, la validación, la documentación y la seguridad no son aspectos aislados, sino partes fundamentales de un mismo sistema. Además, reconocimos que el despliegue es tan importante como el desarrollo, ya que no tiene sentido tener una aplicación funcional si luego es difícil implementarla. Este trabajo nos ayudó a pensar de forma más ordenada y a darle mayor importancia a la calidad, la escalabilidad y la mantenibilidad del software.

Bibliografía

Dokploy. (s. f.). Dokploy documentation. Recuperado el 17 de abril de 2026, de <https://docs.dokploy.com/>

Express. (s. f.). Routing. Recuperado el 17 de abril de 2026, de

<https://expressjs.com/en/guide/routing.html>

MDN Web Docs. (2025). HTTP request methods. Recuperado el 17 de abril de 2026, de

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>

MDN Web Docs. (2025). HTTP response status codes. Recuperado el 17 de abril de 2026, de

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>

OWASP Foundation. (2023). API Security Top 10 2023. Recuperado el 17 de abril de 2026, de

<https://owasp.org/www-project-api-security/>

Railway. (s. f.). Railway documentation. Recuperado el 17 de abril de 2026, de

<https://docs.railway.com/>